Práctica JavaScript — Gestor de Incidencias Técnicas

Contexto

Una empresa necesita una pequeña aplicación web para registrar y consultar incidencias técnicas de sus equipos.

La aplicación tendrá varias páginas y toda la lógica deberá realizarse con **JavaScript**, trabajando con funciones, arrays, objetos, DOM y `sessionStorage`.

Objetivo

Crear una aplicación web que permita:

- Registrar nuevas incidencias.
- Consultar todas las incidencias registradas.
- Filtrar incidencias por prioridad y estado.
- Buscar incidencias por texto.
- Cambiar el estado de una incidencia.
- Eliminar incidencias.
- Mostrar estadísticas básicas.
- Mantener los datos mientras la pestaña siga abierta.

Los datos deben desaparecer al cerrar la pestaña del navegador.

Estructura mínima del proyecto

```
gestor-incidencias/
│
├── index.html
├── crear.html
├── consultar.html
│
├── css/
│   └── style.css
│
└── js/
    ├── constanjkses.js
    ├── storage.js
    ├── crear.js
    └── consultar.js
```

Requisitos generales

- No utilizar clases.
- No utilizar variables globales salvo las constantes necesarias.
- Las constantes deben estar en el fichero `constanjkses.js`.
- Utilizar funciones para dividir la lógica.
- Utilizar `sessionStorage` para almacenar los datos.
- Los datos deberán almacenarse en formato JSON.
- No duplicar código innecesariamente.
- Utilizar nombres de variables y funciones descriptivos.
- El HTML, CSS y JavaScript deberán estar separados.
- Utilizar `type="module"` para cargar los ficheros JavaScript cuando sea necesario.

Datos de una incidencia

Cada incidencia tendrá la siguiente información:

```
id
titulo
descripcion
equipo
prioridad
estado
fecha
```

Ejemplo:

```
{
    id: 1,
    titulo: "El ordenador no arranca",
    descripcion: "Al pulsar el botón no aparece imagen",
    equipo: "PC-23",
    prioridad: "alta",
    estado: "pendiente",
    fecha: "2026-11-02"
}
```

Prioridades permitidas

```
baja
media
```

```
alta
```

Estas opciones deben declararse como constantes en `constanjkses.js`.

Estados permitidos

```
pendiente
en_proceso
resuelta
```

También deberán declararse como constantes.

Página 1 — Home

`index.html`

La página principal mostrará:

- Título de la aplicación.
- Una pequeña descripción.
- Un botón/enlace para crear una incidencia.
- Un botón/enlace para consultar incidencias.
- Un bloque de estadísticas.

Las estadísticas deberán mostrar:

```
Total de incidencias
Pendientes
En proceso
Resueltas
```

Ejemplo:

```
Total: 8
Pendientes: 3
En proceso: 2
Resueltas: 3
```

Las estadísticas deben calcularse usando JavaScript a partir de los datos almacenados.

Página 2 — Crear incidencia

crear.html

Debe contener un formulario con los siguientes campos:

Título

```
<input type="text">
```

Obligatorio.

Descripción

```
<textarea></textarea>
```

Obligatoria.

Equipo

Ejemplo:

```
PC-01
PORTATIL-04
IMPRESORA-02
```

Campo de texto obligatorio.

Prioridad

Utilizar un `<select>` con:

```
Baja
Media
Alta
```

Fecha

Utilizar:

```
<input type="date">
```

Estado

Cuando se crea una nueva incidencia, su estado inicial será siempre:

```
pendiente
```

El usuario no debe elegirlo en el formulario.

Botón

```
<input type="submit" value="Crear incidencia">
```

Funcionamiento de `crear.js`

Al enviar el formulario:

1. Evitar la recarga automática con `preventDefault()`.
2. Leer todos los campos.
3. Validar los datos.
4. Crear un objeto literal JavaScript.
5. Generar un `id` único.
6. Recuperar las incidencias existentes.
7. Añadir la nueva incidencia.
8. Guardar el array actualizado en `sessionStorage`.
9. Mostrar un mensaje confirmando la creación.
10. Limpiar el formulario.

Generación del ID

No se puede escribir el ID manualmente.

Debe generarse automáticamente.

Ejemplo:

```
Primera incidencia  -> id 1
Segunda incidencia  -> id 2
Tercera incidencia  -> id 3
```

Si se elimina una incidencia, no se deben reutilizar IDs antiguos.

Ejemplo:

```
IDs existentes:

1
2
4

Nueva incidencia:

5
```

Página 3 — Consultar incidencias

consultar.html

Debe mostrar todas las incidencias almacenadas.

Cada incidencia aparecerá en una tarjeta.

Ejemplo:

```
---------------------------------
Problema de conexión

Equipo: PC-12
Prioridad: Alta
Estado: Pendiente
Fecha: 02/11/2026

No conecta con la red interna.

[Pasar a En proceso]
[Marcar como resuelta]
[Eliminar]
---------------------------------
```

Crear las tarjetas con JavaScript

No se pueden escribir manualmente las incidencias en el HTML.

Deben generarse utilizando JavaScript.

Se puede utilizar:

```
document.createElement()
```

o:

```
innerHTML
```

El alumno deberá decidir qué opción utilizar.

Filtros

La página de consulta tendrá un filtro por prioridad:

```
Todas
Baja
Media
Alta
```

Y otro filtro por estado:

```
Todos
Pendiente
En proceso
Resuelta
```

Cuando cambie un filtro, solo deben mostrarse las incidencias correspondientes.

Ejemplo:

```
Prioridad: Alta
Estado: Pendiente
```

Debe mostrar solamente las incidencias que cumplan ambas condiciones.

Buscador

Debe existir un campo:

```
<input type="search" id="busqueda">
```

El usuario podrá escribir texto.

La aplicación deberá buscar coincidencias en:

```
titulo
descripcion
equipo
```

La búsqueda no debe distinguir mayúsculas y minúsculas.

Ejemplo:

```
Búsqueda: "portatil"
```

debe encontrar:

```
PORTATIL-04
Portatil dirección
problema en portátil
```

Cambio de estado

Cada tarjeta tendrá botones para cambiar su estado.

Ejemplo:

```
Pendiente
    ↓
En proceso
    ↓
Resuelta
```

Cuando se cambie el estado:

1. Buscar la incidencia por `id`.
2. Modificar su propiedad `estado`.
3. Guardar de nuevo el array en `sessionStorage`.
4. Actualizar las tarjetas mostradas.

No es necesario recargar la página.

Eliminar incidencia

Cada incidencia tendrá un botón:

```
Eliminar
```

Al pulsarlo:

1. Obtener el `id` de la incidencia.
2. Eliminarla del array.
3. Guardar el nuevo array.
4. Volver a mostrar las incidencias.

Estadísticas

Crear una función que reciba el array de incidencias y calcule:

```
total
pendientes
en proceso
resueltas
prioridad alta
```

Ejemplo de resultado:

```
{
    total: 10,
    pendientes: 4,
    enProceso: 3,
    resueltas: 3,
    alta: 2
}
```

Fichero `storage.js`

Este fichero contendrá funciones relacionadas únicamente con el almacenamiento.

Como mínimo:

```
obtenerIncidencias()
guardarIncidencias(incidencias)
obtenerSiguienteId()
```

Ejemplo de responsabilidad:

```
crear.js
    ↓
llama a guardarIncidencias()

consultar.js
    ↓
llama a obtenerIncidencias()
```

Ni `crear.js` ni `consultar.js` deberían contener directamente toda la lógica de `sessionStorage`.

Fichero `constanjkses.js`

Debe contener las constantes utilizadas por la aplicación.

Por ejemplo:

```
export const CLAVE_INCIDENCIAS = "incidencias";

export const ESTADO_PENDIENTE = "pendiente";
export const ESTADO_EN_PROCESO = "en_proceso";
export const ESTADO_RESUELTA = "resuelta";

export const PRIORIDAD_BAJA = "baja";
export const PRIORIDAD_MEDIA = "media";
export const PRIORIDAD_ALTA = "alta";
```

Se pueden añadir otras constantes si son necesarias.

Funciones recomendadas

No es obligatorio utilizar exactamente estos nombres, pero la aplicación debería estar dividida en funciones similares.

`storage.js`

```
obtenerIncidencias()
guardarIncidencias()
obtenerSiguienteId()
```

crear.js

```
crearIncidencia()
validarFormulario()
limpiarFormulario()
mostrarMensaje()
```

consultar.js

```
mostrarIncidencias()
crearTarjeta()
filtrarIncidencias()
buscarIncidencias()
cambiarEstado()
eliminarIncidencia()
```

Validaciones

No se podrá crear una incidencia si:

- El título está vacío.
- La descripción está vacía.
- El equipo está vacío.
- No se ha elegido una prioridad válida.
- No se ha indicado una fecha.

Si hay errores, debe mostrarse un mensaje al usuario.

Parte adicional — Ordenación

Añadir un selector que permita ordenar las incidencias por:

```
Fecha
Prioridad
Estado
```

Para prioridad se utilizará el siguiente orden:

```
Alta
Media
Baja
```

Parte adicional — Contador visual

En la página de consulta debe aparecer:

```
Mostrando X de Y incidencias
```

Ejemplo:

```
Mostrando 3 de 12 incidencias
```

Si se aplica un filtro o búsqueda, `X` deberá actualizarse.

Parte adicional — Incidencias críticas

Una incidencia se considera crítica cuando:

```
prioridad === "alta"
```

y además:

```
estado !== "resuelta"
```

Las incidencias críticas deberán tener una clase CSS adicional:

```
.critica
```

para destacarlas visualmente.

La clase debe añadirse desde JavaScript.

Casos de prueba

Caso 1

Crear:

```
Título: No funciona internet
Equipo: PC-03
Prioridad: Alta
Estado inicial: Pendiente
```

Debe aparecer en consultar.

Caso 2

Crear tres incidencias:

```
Incidencia 1 -> Alta
Incidencia 2 -> Baja
Incidencia 3 -> Alta
```

Filtrar por:

```
Alta
```

Deben aparecer únicamente 2.

Caso 3

Buscar:

```
impresora
```

Solo deben aparecer incidencias que tengan esa palabra en:

```
título
descripción
equipo
```

Caso 4

Cambiar una incidencia:

```
Pendiente -> En proceso
```

Cambiar de página y volver.

Debe seguir apareciendo como:

```
En proceso
```

Caso 5

Eliminar una incidencia.

Cambiar de página y volver.

La incidencia no debe volver a aparecer.

Restricciones

No se permite utilizar:

```
class
new
localStorage
```

Sí se permite utilizar:

```
sessionStorage
arrays
objetos literales
funciones
Map
Set
filter()
find()
findIndex()
sort()
for
for...of
JSON.stringify()
JSON.parse()
```

Entrega

El proyecto deberá subirse a GitHub.

El repositorio deberá incluir:

```
index.html
crear.html
consultar.html
css/
js/
README.md
```

El `README.md` deberá explicar brevemente:

- Qué hace la aplicación.
- Cómo está organizada.
- Qué datos se almacenan.
- Qué funciones principales se han implementado.